Enterprise Order Processing Platform

Project report: what was built, how it works, and how it was tested, measured and deployed

Author	Khushi Desai (GitHub: khushidesai23)
Period	9 July 2026 to 28 October 2026
Repository	github.com/khushidesai23/Enterprise-Order-Processing - main branch `develop`, state described: `feature/load-testing` at 446347a
In one sentence	An online-shop backend in Go that takes orders, reserves stock safely under concurrency, collects payments through Razorpay, streams database changes to Kafka, and is fully observable, load-tested and deployable, with a React storefront and admin console on top.

337 commits	**18** pull requests	**~50** REST endpoints	**8** business modules	**92+18+12** Go / browser / UI tests
17.7k lines of Go (app)	**4.5k** lines of Go tests	**9.2k** lines of React/TS	**11** Docker services	**11** load-test scenarios

Contents

- 1. Project overview and timeline
- 2. Architecture
- 3. Backend features, module by module
- 4. Data model and consistency rules
- 5. Security and access control
- 6. Frontend: meridian/ storefront and admin console
- 7. Logging, metrics and tracing (observability)
- 8. Change data capture with Debezium and Kafka
- 9. Testing and continuous integration
- 10. Load testing and performance work
- 11. Failure and recovery drills
- 12. Deployment
- 13. Developer tooling and documentation
- 14. Known limitations and next steps
- Appendix: technology stack

1. Project overview and timeline

The project is a production-style order processing system for an online store. A customer browses a catalogue, puts items in a bag, places an order (which reserves stock), pays with Razorpay, and follows the order through packing, shipping and delivery. Admins manage the catalogue, stock, orders, payments, refunds, reviews and customers.

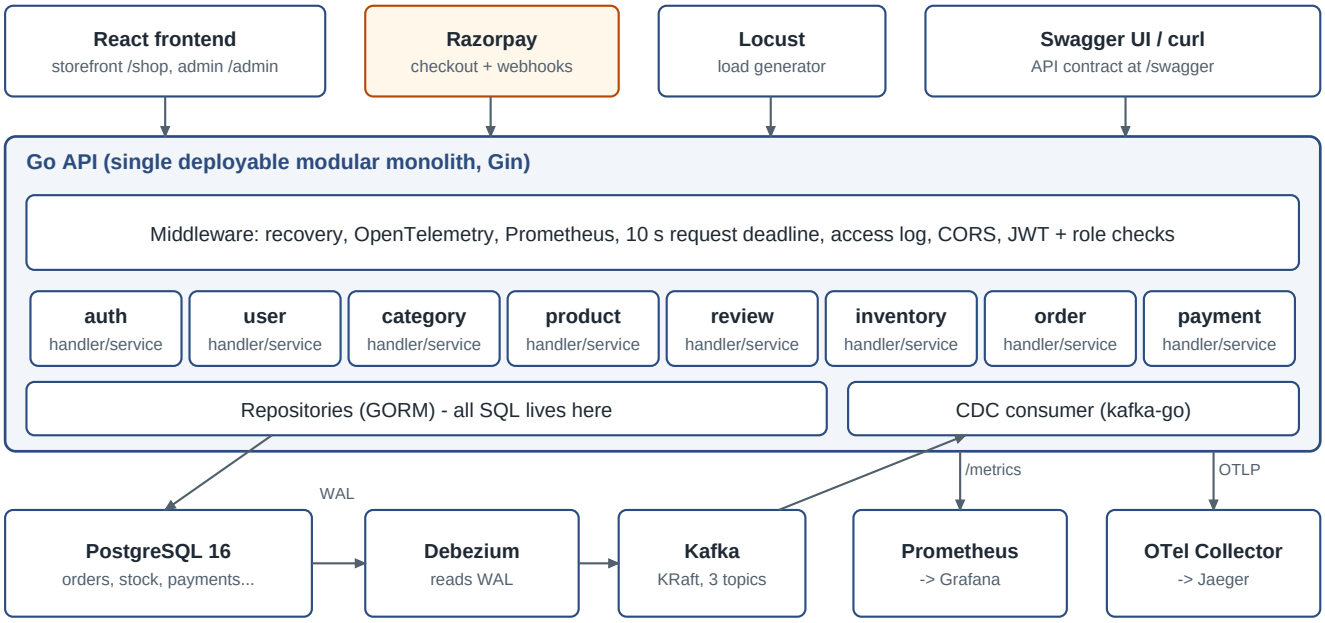
It was built as a learning-oriented but realistic system, so the goal was not only features but the engineering around them: correctness under concurrency, idempotent payment handling, role-based security, observability, automated tests, load and failure testing, and a real deployment. Work was done on feature branches and merged into `develop` through pull requests, each with a focused scope.

How the work progressed

When (2026)	Branch / PR	What was delivered
9-15 Jul	initial setup	Go module, Docker PostgreSQL, Makefile, configuration (Viper, .env), Zap logger, health and ping endpoints, graceful shutdown, standard API responses, password hashing, GORM models and migrations.
15-21 Jul	core modules	Users, products, categories and inventory with full CRUD; repository/service/handler layers; orders with transactional stock reservation and status updates.
22-30 Jul	#1-#4 payments, Swagger	Razorpay gateway abstraction, payments, webhook model with signature verification and event-ID idempotency; Swagger/OpenAPI docs for every endpoint; CORS; a local Razorpay checkout demo page.
3-7 Aug	#5-#6 authentication	JWT login, auth middleware, protected routes, orders bound to the logged-in user; README and roadmap.
10-17 Aug	#7 otel, #8 locust	OpenTelemetry tracing (HTTP + SQL spans) to an OTel Collector and Jaeger; context passed through every layer; trace-correlated logs; first Locust scripts.
19-21 Aug	#9 testing-ci, #10 prometheus, #11 grafana	Unit tests with mocked repositories, DB integration tests, HTTP end-to-end tests, GitHub Actions CI; Prometheus metrics and middleware; Grafana.
22-25 Sep	#12-#13 cdc	PostgreSQL logical replication, Debezium connector, Kafka (KRaft), and a Go CDC consumer with retry/backoff and metrics.
9 Oct	load testing PR	Locust scenario suite (11 scenarios), seeded test data, provisioned Grafana dashboard, DB pool metrics; fixes found under load: atomic stock updates, deadlock-free locking, webhook hardening, 14 to 5 SQL statements per order, 10 s request deadline, pool sizing, login errors.
21 Oct	#15 rbac	customer/admin roles in the JWT, `RequireRole` middleware, resource ownership checks (Principal), admin promotion, role-aware e2e tests.
23 Oct	#16 payment-lifecycle	Refunds, checkout verification endpoint (works without a public webhook URL), out-of-order and duplicate webhook rules, order status summary and filters, soft-delete fixes; 19 lifecycle e2e tests.
23-26 Oct	#17-#18 frontend	React storefront and admin console, product search with facets, reviews, demo-store seeder (840 products, 25,000 orders), Playwright tests, the meridian/ redesign, and Netlify + Render + Neon deployment.
28 Oct	feature/load-testing	Failure and recovery drills (separate report): crash, outage, timeout, duplicate-delivery, overload and backup/restore tests.

2. Architecture

The backend is a **modular monolith**: one Go program, deployed as one unit, but split internally into business modules that only talk to each other through Go interfaces. This keeps local development and deployment simple, while any module could later be pulled out into its own service without changing its callers.



Every request follows the same path: **router -> middleware -> handler -> service -> repository -> PostgreSQL**.

Layer	Responsibility	Where
Router and wiring	Builds every dependency and registers routes; each module exposes `RegisterRoutes`	`internal/app`, `internal/api/routes`
Middleware	Panic recovery, tracing, metrics, 10 s request deadline, access log, CORS, JWT authentication, role checks	`internal/api/middleware`
Handler	Parses and validates HTTP input, calls the service, writes a standard JSON response	`internal/modules/*/handler.go`
Service	Business rules and transactions (e.g. order creation, payment state machine)	`internal/modules/*/service.go`
Repository	All database access through GORM; row locks and guarded updates	`internal/repository`
Models	Tables as Go structs with UUID keys, timestamps and soft delete	`internal/models`
Shared packages	Logger, Prometheus metrics, OpenTelemetry tracer, password hashing, validation	`pkg/`

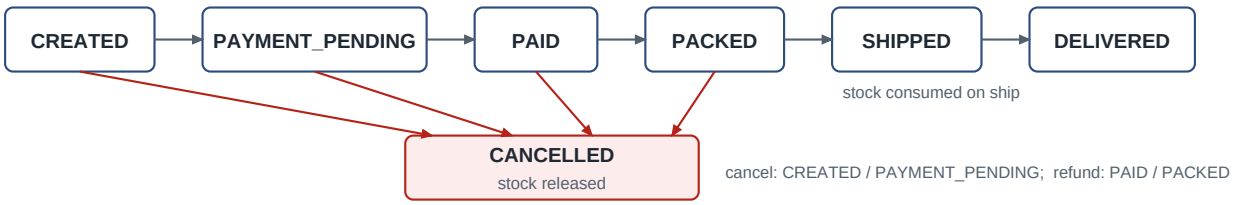
Runtime components (docker-compose.yml)

Service	Purpose
postgres (16)	Main database, with logical WAL enabled for change data capture
postgres-test	Separate database used only by integration and end-to-end tests (port 5433)
kafka (3.2, KRaft)	Event stream; no ZooKeeper
debezium (Connect 3.2)	Reads the PostgreSQL WAL and publishes table changes to Kafka
otel-collector, jaeger	Receives OpenTelemetry traces and stores them for search
prometheus, grafana	Scrapes metrics every 5 s; provisioned dashboard and Jaeger data source
API (host or Docker)	The Go server; also a `Dockerfile` that builds the server and the seeder

3. Backend features, module by module

Module	What it does
Auth	Email/password login returns a signed JWT (HS256, 24 h default) carrying the user ID and role. `GET /auth/me` returns the current user. Unknown emails get the same 401 as a wrong password, with a dummy bcrypt compare so timing does not reveal which emails exist.
Users and roles	Self sign-up always creates a `customer`. Users read and edit their own profile; admins list, deactivate, delete and change roles (`PATCH /users/:id/role`), but not their own. The first admin is promoted with `make promote-admin`.
Categories	CRUD for departments. A category that still has products cannot be deleted. Names of soft-deleted categories can be reused.
Products and search	CRUD with SKU, brand, price and MRP, images and specs. `GET /products/search` offers word search, filters (category, brands, price range, rating, in stock, deals), sorting, paging and facet counts used by the storefront filters.
Reviews	One review per customer per product (create or edit with `PUT`), marked *verified* when the customer bought the product; public listing with a rating histogram; authors or admins can delete; product ratings are recalculated.
Inventory	Per-product available and reserved quantities. Admin operations: add or remove stock, set levels, reserve, release and confirm. Every decrement is a single guarded `UPDATE ... WHERE available_quantity >= ?`, so stock can never go negative.
Orders	Create an order from items; the total is computed from current prices and stock is reserved in the same transaction. Customers list, read and cancel their own orders; admins list all orders with paging and status filters, see a per-status summary, and move orders through fulfilment (packed, shipped, delivered).
Payments	Checkout creates a Razorpay order and a PENDING payment. The browser confirms with `POST /payments/verify` (HMAC signature check) and Razorpay webhooks are the backstop; both are idempotent with each other. Failed payments can be retried on the same Razorpay order. Admins issue refunds (a refund before shipping also cancels the order and releases stock). A payment never moves backwards.
Health	`/health` (status + DB), `/ready` (503 when the DB is unreachable), `/ping`, `/version`; used by Render's health check and the storefront footer.

Order lifecycle



Payment and order states change only in defined ways. `PAYMENT_PENDING` and `PAID` are set only by the payment flow; the admin status endpoint handles fulfilment and cancellation of unpaid orders. A paid order is cancelled by refunding it, which returns the money as well.

API surface

Group	Endpoints (all under /api/v1)
Auth	POST /auth/login, GET /auth/me
Users	POST /users (sign-up), GET /users, GET/PUT/DELETE /users/:id, PATCH /users/:id/role
Catalogue	/categories (CRUD), /products (CRUD), GET /products/category/:id, GET /products/search
Reviews	GET /products/:id/reviews, GET+PUT /products/:id/reviews/me, DELETE /reviews/:id, GET /reviews (admin)
Inventory	/inventory, /inventory/:productId, add-stock, remove-stock, reserve, release, confirm
Orders	POST /orders, GET /orders (paging, status filter), /orders/summary, /orders/me, /orders/:id, PATCH status, PATCH cancel
Payments	POST /payments, POST /payments/verify, GET /payments, /payments/summary, /payments/:id, /payments/order/:orderId, POST /payments/:id/refund, POST /payments/webhook
Operations	/health, /ready, /ping, /version, /metrics, /swagger/index.html

Every response uses one JSON envelope (`success, message, data | error, timestamp`), and the full contract is generated as Swagger/OpenAPI from code annotations.

4. Data model and consistency rules

Table	Key contents and constraints
users	email (unique among non-deleted), bcrypt password hash, role (customer/admin), is_active
categories, products	name / SKU unique among non-deleted rows; product price, MRP, brand, images, specs, rating
inventories	one row per product: available_quantity, reserved_quantity
orders, order_items	user, status, total; items keep the price at the time of purchase
payments	one per order; unique gateway_order_id and transaction_id; status PENDING / SUCCESS / FAILED / REFUNDED
payment_webhooks	every received Razorpay event; unique payload_id (the event ID) is the idempotency key; raw JSON payload kept
reviews	one per (user, product); rating, text, verified flag

All tables use UUID primary keys, created/updated timestamps and soft delete. The schema is created by GORM AutoMigrate at start-up.

Consistency rules (the core of the project)

- **One transaction per order.** Insert the order, reserve stock for each item, insert the items, commit. If any step fails nothing is kept. Prices and the user are read before the transaction so locks are held only for the writes.
- **Stock can never go negative.** A reservation is one guarded UPDATE that both checks and changes the quantity, so two buyers of the last unit cannot both succeed.
- **No deadlocks.** Items are locked in product-ID order, and every payment path locks the order row first, so concurrent operations always take locks in the same order.
- **Status changes are serialized.** Cancel, payment, shipping and refund lock the order row (`SELECT ... FOR UPDATE`), so stock is released or consumed exactly once.
- **Webhooks are idempotent and order-safe.** Duplicate event IDs are acknowledged and ignored, even when delivered concurrently; a failed application rolls back the webhook record so Razorpay's retry is processed; a late `payment.failed` cannot undo a capture and a capture cannot undo a refund.
- **Bounded requests.** Each request has a 10 s deadline (below the 15 s server write timeout); stalled queries are cancelled and the client receives 504 instead of a dropped connection.

> These rules were proven by tests rather than assumed: the integration suite runs 50 concurrent orders against 10 units of stock, concurrent cancels, opposite-order multi-item orders and concurrent stock removals, and the contention load test ended with exactly 100 units sold in 100 orders.

5. Security and access control

Concern	How it is handled
Passwords	bcrypt hashes; never returned by the API
Authentication	JWT bearer tokens signed with a secret from the environment; the app refuses to start without `JWT_SECRET` and Razorpay keys
Authorization	`RequireRole(admin)` on every admin route, plus ownership checks in services through an `authz.Principal`; the UI hiding admin pages is cosmetic, the API enforces it
ID probing	Asking for someone else's order, payment or user returns 404, identical to a missing record
Payments	Checkout and webhook signatures verified with HMAC-SHA256 (key secret / webhook secret); bad signature returns 401
CORS	Explicit list of allowed browser origins (`CORS_ALLOWED_ORIGINS`)
Secrets	`.env` is git-ignored and excluded from the Docker image; hosted secrets live in the Render and Netlify dashboards
Tracing privacy	SQL spans are recorded without bind values

6. Frontend: meridian/ storefront and admin console

One React application (`frontend/`) with two areas: the public storefront at `/shop` and the admin console at `/admin`. It is built with React 18, TypeScript, Vite 6, Tailwind CSS 4, Radix UI primitives, React Router 6, TanStack Query 5 for server data, Recharts for charts and Motion for animation. All pages use live API data; the dev server proxies `/api` to the Go API so no CORS setup is needed locally.

Storefront

- **Design.** An art-directed identity called *meridian/*, an "index of objects": warm paper and ink colours with an acid-lime signal colour, Syne, Instrument Serif, Geist and Geist Mono type, a globe-and-meridian logo. Motion includes word reveals, a CSS 3D orbit of best sellers, a scroll-driven gallery, magnetic buttons and a fly-to-bag animation; everything respects reduced-motion settings.
- **Home:** live catalogue counts, today's biggest real discounts, department index, each department's best seller, quotes from real reviews, new arrivals.
- **Search:** a command palette (Ctrl+K or /) and a full search page with department, brand, rating, price, deals and in-stock filters with counts, sorting and paging; filters live in the URL.
- **Product pages:** zoom gallery, real variant options with live stock, stock meter, quantity stepper, Add to bag and Buy now, specs, related products and reviews.
- **Bag, wishlist, recently viewed:** bag checked against live stock and prices; save for later; stored in the browser.
- **Checkout and payment:** create the order, open Razorpay Checkout, confirm with the verify endpoint; the UI says "It's yours" only after the backend reports PAID; double submissions are blocked.
- **Orders and account:** order history, an animated tracker (placed, paid, packed, shipped, delivered), Pay now / Retry payment and Cancel for unpaid orders, sign-up and sign-in that return you to the page you were on.
- **Health indicator** in the footer from `GET /api/v1/health`.

Admin console

- **Overview:** revenue, order, low-stock and customer KPIs, 14-day captured-revenue chart, orders by status, recent orders, low-stock warnings.
- **Catalogue:** products (create with starting stock, edit, delete, search, filter) and categories.
- **Inventory:** add/remove/set stock, reserve, release, confirm, low and out-of-stock filters.
- **Orders:** status tabs, order detail, fulfilment (packed, shipped, delivered), cancellation of unpaid orders, refunds.
- **Payments, reviews, customers:** payment summary and refunds; review moderation; customer search, admin grant/removal, deactivation and deletion.

Pages: 10 storefront, 9 admin, 2 auth, plus layouts and a shared component library (`src/components/ui`, `src/store`).

7. Logging, metrics and tracing (observability)

Three signals are produced and linked together, so a slow request seen on a dashboard can be followed to its trace and its log lines.

Logging

- **Zap** structured JSON logs on stdout with ISO-8601 timestamps; the level is set by `LOG_LEVEL`.
- **Trace correlation:** helpers (`logger.InfoContext`, `WarnContext`, `ErrorContext`) add the OpenTelemetry `trace_id` and `span_id` to every log line written during a request.
- **Access log middleware:** one line per request with method, path, status, latency, body size and client IP.
- **Business events** are logged with their IDs: order created, payment created/completed/retried, webhook received/processed/ignored, and warnings such as "payment captured for cancelled order; refund required".
- Infrastructure errors (pool exhausted, timeouts) are logged and returned as 500/504 rather than being disguised as 401.

Metrics (Prometheus, scraped every 5 s)

Metric	Meaning
`order_processing_http_requests_total`	Requests by method, route template and status
`order_processing_http_request_duration_seconds`	Latency histogram per route (P50/P95/P99 in Grafana)
`order_processing_http_requests_in_flight`	Requests currently being handled
`go_sql_*`	Connection pool: open, in use, idle, waits and wait time, closes by limit
`order_processing_cdc_*`	CDC events consumed, consumer errors, end-to-end lag, consumer lag
`locust_*`	Load generator users, requests and latency, plotted next to server metrics
`go_*, process_*`	Go runtime: goroutines, memory, GC, CPU

Tracing

OpenTelemetry is initialised at start-up; `otelgin` creates a span per HTTP request and `otelgorm` a child span per SQL statement. The request context is passed through every handler, service and repository so spans nest correctly. Spans go over OTLP to the OTel Collector and on to Jaeger (service `enterprise-order-processing`). `/metrics` scrapes are excluded from metrics, traces and access logs so they do not pollute the data.

Dashboard

Grafana is provisioned from the repository (no manual setup): the **Order Processing - Load Testing** dashboard shows load generator vs server traffic, latency percentiles overall and per route, errors by status, Go runtime, DB pool, CDC lag and a slow-trace table linked to Jaeger.

8. Change data capture with Debezium and Kafka

Instead of having the API publish events itself (which risks writing to the database but failing to publish, or the reverse), the project uses **change data capture**: PostgreSQL writes every committed change to its write-ahead log (WAL); Debezium reads the WAL through a replication slot and publishes each row change to a Kafka topic (`order-processing.public.orders`, `.inventories`, `.categories`). Events therefore exist if and only if the transaction committed.

A Go consumer inside the API (kafka-go, consumer group) reads those topics, decodes the Debezium envelope (create, update, delete, snapshot), records metrics including commit-to-consume lag, and retries read errors with exponential backoff (1 s to 30 s). It currently counts and logs events: it is the foundation for future notification, analytics and audit consumers. Measured steady-state lag was 1-5 s at P95 with 50-74 events/s. CDC can be switched off with `KAFKA_CDC_ENABLED=false` (used in hosting).

9. Testing and continuous integration

Testing is layered so that fast tests cover the logic and slower tests prove the real database, HTTP and browser behaviour.

Layer	Count	What it covers	Runs
Go unit tests	65 test functions in 19 files	Services with mocked repositories (order, payment, inventory, product, category, user, auth), JWT, passwords, middleware (auth, roles, CORS, timeout, Prometheus), routes, handlers, signature and webhook parsing, authorization rules	`go test ./...`
Go integration tests	5 functions (many sub-tests)	Real PostgreSQL: repositories, transactions, constraints, and concurrency regressions: 50 concurrent orders for 10 units, concurrent cancels, opposite-order multi-item orders (deadlock), concurrent stock removals	`-tags=integration`, test DB
Go HTTP end-to-end	22 functions	Full HTTP stack with a stub gateway: checkout flow, search and facets, reviews, 19 payment-lifecycle cases (retry, abandon, cancel, refund before/after shipping, refund rollback on gateway failure, webhook redelivery, concurrent webhooks, failed processing retried, capture after cancel, verification)	`-tags=e2e`, test DB
Frontend unit	12 tests	API client, cart, order status rules, formatting (Vitest)	`npm test`
Frontend type check	whole app	TypeScript, including the browser test specs	`npm run typecheck`
Browser end-to-end	18 Playwright tests	Real UI against real API and DB: registration, search, bag, checkout, payment, retry, cancel, reviews, wishlist, admin catalogue, inventory, fulfilment, refunds, moderation, customers, access control	`npx playwright test`
Load tests	11 Locust scenarios	Performance and correctness under load (section 10)	Locust
Failure drills	9 drills	Crash, outage, timeout, duplicates, overload, backup/restore (section 11)	drill harness

Continuous integration (GitHub Actions)

On every push and pull request: **backend job** - gofmt check, build, unit tests, repository and order-service integration tests, HTTP end-to-end tests, against a PostgreSQL 16 service container; **frontend job** - npm ci, type check, unit tests, production build.

Test results for this report (28 Oct 2026, local)

Suite	Result
Go unit tests (go test ./...)	**PASS - all 10 packages with tests (result served from Go's test cache: code unchanged since the last run)**
Repository integration (real PostgreSQL)	**PASS - fresh run, 28.3 s**
Order service integration (real PostgreSQL)	**PASS - fresh run, 8.3 s**
HTTP end-to-end (real PostgreSQL, stub gateway)	**PASS - fresh run, 54.4 s**
Frontend type check (tsc -b)	**PASS - no errors**
Frontend unit (Vitest)	**PASS - 12 of 12**
Browser end-to-end (Playwright)	Not run for this report: it creates real Razorpay test-mode orders over the internet and needs an admin account

Integration and end-to-end suites ran with -count=1 against the postgres-test container (port 5433), which they truncate; the development database was not touched.

10. Load testing and performance work

A Locust suite drives realistic traffic. `locust/seed.py` creates a controlled data set (a load-test customer, 10 products with 1,000,000 units, a 100-unit contention product), every scenario checks stock before starting so inventory exhaustion cannot distort results, and a Prometheus exporter puts Locust's view next to the server's in Grafana.

Scenario	Users	Purpose
Smoke, baseline	2, 5	Wiring, metrics and traces; mixed reads and writes at low load
Read-heavy, mixed	50	Catalogue browsing with some order traffic
Order-heavy	50	Write path: one-unit orders across 10 products
Inventory contention	100	Everyone buys one 100-unit product; checks no overselling
Auth	25	Login cost (bcrypt) in isolation
Spike, stress, recovery, sustained	up to 400	Sudden surges, step increases to saturation, recovery after overload, 15 minutes for leaks

Selected results (local machine)

Run	Users	Req/s	P50 / P95 / P99 ms	Failures
Mixed	50	24.6	6 / 27 / 91	0 of 5,907
Order-heavy, before optimisation	50	22.0	57 / 1,100 / 3,300	0
Order-heavy, after optimisation	50	23.5	13 / 420 / 2,700	0
Stress to 400 users	400	86.8	15 / 110 / 1,000	12 of 93,768 (disk stall)
Spike 20-300-20, after fixes	300	61.1	13 / 150 / 1,300	0 of 29,326
Sustained 15 min	100	48.0	8 / 74 / 1,400	65 of 43,178 (disk stalls); no memory or goroutine growth

What load testing found and fixed

- **Wrong database:** the API was writing to a native Windows PostgreSQL, not the container Debezium reads; moved to port 5434.
- **Races:** concurrent cancels could release stock twice; multi-item orders deadlocked. Fixed with row locks, stable lock order and guarded updates.
- **Slow orders:** CreateOrder went from 14 to 5 SQL statements; P50 56 to 13 ms, P95 1,100 to 430 ms. The remaining tail is disk fsync on Docker Desktop (proved with a temporary synchronous_commit=off test, then reverted to keep durability).
- **Silent dropped connections:** handlers outlived the 15 s write timeout during stalls; added the 10 s request deadline so clients get 504.
- **Pool larger than the server allowed** (100 vs max_connections 100 plus Debezium) caused "too many clients"; pool made configurable, default 80, idle churn removed.
- **Login errors:** any lookup failure became 401 and an unknown email panicked; now 401 vs 500/504 correctly, with constant-time comparison.
- **Measurement fixes:** dashboards mixed in Prometheus' own metrics, `/metrics` scrapes were counted as traffic; both fixed.

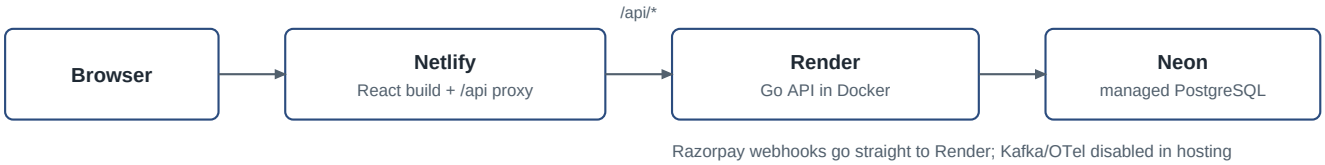
11. Failure and recovery drills

Nine bounded drills on the local stack asked what breaks when a component stops, slows down, or delivers the same work twice. Full details are in the separate report `documentation/FAILURE_RECOVERY_DRILLS.pdf`.

Held up	Needs work
No lost or half-written orders across ~40,000 orders through an API crash, database stop and hang, Kafka stop and Debezium kill. Webhook de-duplication held under 6 concurrent copies racing 3 browser confirmations. An interrupted webhook was rolled back and applied once on redelivery. No CDC event was lost in any outage. A backup taken under load restored consistently in 12 s.	Razorpay call ignores the request deadline and holds the order lock; POST /orders has no idempotency key, so a retry after an ambiguous error duplicates the order; Debezium does not recover by itself after a database restart or Kafka outage; outages return 500 instead of 503; no load shedding; backups are manual only.

12. Deployment

The project is deployable on free tiers of three platforms, with the configuration committed to the repository.



Piece	Platform	Committed configuration
Frontend	Netlify	`netlify.toml`; a build script writes the `/api` proxy and SPA fallback from `API_ORIGIN`; caching and security headers in `_headers`
API	Render (Docker)	`Dockerfile` (server + seeder), `render.yaml` blueprint with health check and environment variables; JWT secret generated
Database	Neon PostgreSQL	Direct (non-pooled) connection for GORM prepared statements; schema created by AutoMigrate; optional demo-store seeding

Kafka, Debezium, Prometheus, Grafana and Jaeger are not deployed; the API runs without them through `KAFKA_CDC_ENABLED=false` and `OTEL_ENABLED=false`. DEPLOYMENT.md walks through every step, including the Razorpay webhook, CORS origin, first admin, and free-tier caveats (Render sleeps after 15 minutes).

13. Developer tooling and documentation

Tool	What it gives
Makefile	run, build, test, test-db, swagger, fmt, tidy, docker-up/down, logs, promote-admin, seed, frontend-* targets
Demo-store seeder (`cmd/seed`)	~840 products with photos in 24 departments (DummyJSON catalogue expanded into variants), 1,500 customers, 25,000 orders with payments over a year, ~11,700 reviews; deterministic and safe to re-run
Swagger / OpenAPI	Generated from annotations; interactive UI at /swagger/index.html
payment-demo/	Stand-alone Razorpay test-mode checkout page for trying payments and webhooks (with ngrok)
Debezium setup	`config/debezium/setup.sql` (publication, slot) and the connector JSON
Configuration	Everything from environment variables (Viper, optional .env); `.env.example` documents every setting

Documentation in the repository

Document	Contents
README.md	Architecture, consistency rules, lifecycle, setup, configuration, API, authorization, payments, observability, testing, limitations, roadmap
frontend/README.md	Frontend setup, demo data, features, payments without a public webhook, tests
LOAD_TESTING.md	Environment, scenarios, what to watch, measured results, findings and fixes
DEPLOYMENT.md	Netlify + Render + Neon step by step

Document	Contents
documentation/	ARCHITECTURE, DEVELOPMENT, OBSERVABILITY, TESTING, ROADMAP, and the failure-drills report

14. Known limitations and next steps

- **Role changes wait for token expiry:** the role is read from the JWT, so a demoted or deactivated user keeps access until the token expires (24 h). Next: short-lived tokens with refresh, or revocation.
- **Refunds:** only full refunds are modelled; partial refunds are acknowledged but not recorded; `refund.failed` is not handled.
- **Capture after cancel** is logged for an admin to refund rather than refunded automatically.
- **CDC consumers have no business logic yet.** Next: notification, analytics and audit consumers with idempotency, commit-after-processing, retries and a dead-letter topic.
- **From the drills:** move the gateway call out of the locked transaction with a shorter timeout; add an Idempotency-Key to order creation; watchdog and lag alerts for Debezium; 503 for outages; load shedding; scheduled backups or point-in-time recovery.
- **Operations:** alerting and SLOs, PostgreSQL and Debezium JMX exporters; TimescaleDB for operational time series is on the roadmap.

Appendix: technology stack

Area	Technologies
Backend	Go 1.26, Gin, GORM (pgx), Viper, Zap, golang-jwt, bcrypt, razorpay-go, Swaggo
Data and events	PostgreSQL 16 (logical WAL), Debezium Connect 3.2, Kafka 3.2 (KRaft), kafka-go
Observability	OpenTelemetry SDK, otelgin, otelgorm, OTel Collector 0.158, Jaeger 1.76, Prometheus 3.11, Grafana 13
Frontend	React 18, TypeScript, Vite 6, Tailwind CSS 4, Radix UI, React Router 6, TanStack Query 5, Recharts, Motion
Testing	Go testing + testify, build-tagged integration and e2e suites, Vitest, Playwright, Locust 2.46
Delivery	Docker, Docker Compose, GitHub Actions, Netlify, Render, Neon